

Advanced SQL

Views, Procedures, Optimization, Design, and
Scaling

Views: The Virtual Table

Key Concepts

- Virtual Table: A stored query that acts like a table.
- No Data Stored: Views dynamically fetch data from underlying tables.
- Security Layer: Restrict access to specific columns or rows.

Code Example

Create a simplified view for non-admin users:

```
CREATE VIEW PublicStudentInfo AS  
SELECT FirstName, LastName, EnrollmentDate  
FROM Students  
WHERE IsActive = 1;
```

Common Table Expressions (CTEs)

Key Concepts

- Named Temporary Result Set: Defined within a single query.
- Exists Only During Query: Not stored persistently.
- Readability: Offers a clearer alternative to nested subqueries.

Code Example

Identify high-performing students:

```
WITH HighPerformers AS (  
  SELECT StudentID, AVG(Grade) as GPA  
  FROM Transcripts  
  GROUP BY StudentID  
  HAVING AVG(Grade) > 3.5  
)  
SELECT s.FirstName, hp.GPA  
FROM Students s  
JOIN HighPerformers hp ON s.StudentID = hp.StudentID;
```

Stored Procedures: Reusable Logic

Key Concepts

- Pre-compiled Code: Optimized for repeated execution.
- Parameters: Accept input values for dynamic behavior.
- Actions: Perform DML operations (INSERT, UPDATE, DELETE).
- Security: Helps prevent SQL injection attacks.

Code Example

Update a student's grade:

```
CREATE PROCEDURE UpdateGrade
    @StudentID INT,
    @CourseID INT,
    @NewGrade CHAR(1)
AS
BEGIN
    UPDATE Enrollments
    SET Grade = @NewGrade, LastUpdated = GETDATE()
    WHERE StudentID = @StudentID AND CourseID = @CourseID;
END;

EXEC UpdateGrade
    @StudentID = 1,
    @CourseID = 101,
    @NewGrade = 'A';
```

Transactions: ACID Properties

Definition

A transaction is a sequence of operations executed as a single, indivisible logical unit of work.

The "ACID" Test

- **Atomicity:** All or nothing. All operations succeed, or all are rolled back.
- **Consistency:** Data remains valid before and after the transaction, adhering to all rules and constraints.
- **Isolation:** Concurrent transactions do not interfere with each other's execution.
- **Durability:** Once committed, changes are permanent and survive system failures.

Transaction Example: The Bank Transfer

Scenario: Transfer \$500 from Account A to Account B, ensuring data integrity.

```
BEGIN TRANSACTION;  
BEGIN TRY  
  -- Step 1: Deduct from Alice  
  UPDATE Accounts SET Balance = Balance - 500 WHERE ID = 1;  
  -- Step 2: Add to Bob  
  UPDATE Accounts SET Balance = Balance + 500 WHERE ID = 2;  
  -- If we get here, both worked. Save it!  
  COMMIT TRANSACTION;  
END TRY  
BEGIN CATCH  
  -- If ANY error happened above, undo everything.  
  ROLLBACK TRANSACTION;  
  PRINT 'Transfer failed. Money restored.';  
END CATCH;
```

Indexes: The Speed Boost

Key Concepts

- Table Scan vs. Index Seek: Indexes allow direct access to data, avoiding full table scans.
- B-Tree Structure: Organized like a library catalog for efficient data retrieval.
- Trade-off: Faster reads at the cost of slightly slower writes (updates, inserts, deletes).

Visual Metaphor

Searching a phone book (indexed A-Z) versus searching through a disorganized pile of individual papers.

Types of Indexes

Key Concepts

- **Clustered Index:** Physically sorts the table data. Typically created automatically on the Primary Key. Only one per table.
- **Non-Clustered Index:** A separate structure containing pointers to the actual data rows, similar to a book's index. Multiple allowed per table.

Code Examples

Clustered (often automatic with PK):

```
ALTER TABLE Students  
ADD CONSTRAINT PK_Students PRIMARY KEY (StudentID);
```

Non-Clustered (for frequent search columns):

```
CREATE INDEX idx_LastName ON Students(LastName);
```

Database Partitioning (Optimization within one Server)

As datasets grow and traffic increases, databases need to scale. We explore two primary strategies to manage large-scale data.

Database Partitioning

Optimization within one server. Splitting a large table into smaller, more manageable pieces *within the same database instance*.

- **Horizontal:** By rows (e.g., Orders_2023).
- **Vertical:** By columns (e.g., separating large blobs).

Benefits: Faster queries (scan relevant partitions), easier maintenance (drop old partitions instantly).

Database Sharding (Scaling Out across Servers)

Scaling out across servers. Distributing data across *multiple separate physical servers* (nodes).

- **Shard Key:** Determines which data goes to which server (e.g., UserID).
- **Infinite Scale:** Add servers as needed.

Challenges: Increased complexity, difficult cross-shard joins, application-level logic required.

Partitioning vs. Sharding (Comparison)

LOCATION

Partitioning: Same Server

Sharding: Multiple Servers

GOAL

Partitioning: Manageability & Query Speed

Sharding: Infinite Storage & Write Throughput

COMPLEXITY

Partitioning: Low (SQL Engine)

Sharding: High (Application/DevOps)

ACID

Partitioning: Easy (Standard Transactions)

Sharding: Hard (Distributed Transactions)

ER Diagrams: Visualizing Data Relationships

- An ERD helps visualize how data connects, ensuring a solid foundation for your database design.
- **Pay close attention to Cardinality**—accurately defining relationships like "One-to-Many" or "Many-to-Many" is crucial.
- Missteps in cardinality can lead to costly refactoring and data inconsistencies later on.

